RÉPUBLIQUE DÉMOCRATIQUE DU CONGO

DOCUMENT 01 · PRÉSENTATION DU PROJET

PHP-SHIELD RDC

Plateforme de gouvernance de la sécurité applicative PHP et de la résilience cybernétique.

Fondateur ultime & propriétaire du projet

LOKALA LOMBOTO Jonathan

VISION · ARCHITECTURE · SÉCURITÉ · FEUILLE DE ROUTE · NOVEMBRE 2026

DOSSIER DE PRÉSENTATION · VERSION PUBLIQUE · 10/11/2026

PHP-SHIELD RDC · DOCUMENTATION INSTITUTIONNELLE

Présentation du projet

Une base de décision claire pour comprendre l'ambition, l'architecture proposée et le niveau de maturité actuel.

> **Statut du document**
> Dossier de présentation d'un projet technique proposé pour évaluation. Il ne constitue ni une décision d'achat, ni une homologation, ni une accréditation et ne revendique pas un déploiement institutionnel.

Fiche d'identité

Élément	Description
Nom	PHP-SHIELD RDC — plateforme de gestion de la sécurité des applications PHP.
Porteur du projet	LOKALA LOMBOTO Jonathan, fondateur ultime et propriétaire de l'application.
Public visé	Équipes techniques, administrateurs et responsables de gouvernance habilités.
Périmètre cible	Actifs applicatifs et environnements explicitement autorisés par leurs responsables.
Édition	Dossier public de présentation — 5 novembre 2026.

Sommaire

01 Synthèse exécutive
02 Contexte et objectifs
03 Périmètre fonctionnel
04 Architecture et flux
05 Données et contrôles de sécurité
06 Maturité et feuille de route
07 Gouvernance et limites
08 Préparation d'un pilote
09 Indicateurs de réussite
10 Glossaire

01 · Synthèse exécutive

PHP-SHIELD RDC vise à fournir un espace de travail centralisé pour inventorier les applications PHP autorisées, comprendre leur environnement technique, organiser l'analyse des dépendances et suivre la correction de constats de sécurité. L'objectif de gestion est concret : lorsqu'une vulnérabilité importante est annoncée, une équipe doit pouvoir repérer rapidement les actifs potentiellement concernés, identifier les interlocuteurs et suivre les actions correctives.

Le dépôt contient aujourd'hui une interface React/Vite, une API FastAPI de fondation, un modèle de données SQLAlchemy, une première migration Alembic, une base PostgreSQL prévue par Docker Compose et un parcours d'aperçu public. Le site GitHub Pages ne sert que le frontend statique. Les fonctions métier de registre et d'audit sont encore à construire; la démonstration ne doit donc pas être interprétée comme une plateforme de scan en production.

> **Positionnement**
> Le projet est une proposition technique portée par le fondateur. Toute adoption, connexion à un système réel, désignation d'autorité ou mise en production nécessite une évaluation et une validation explicites par les organismes compétents.

Résultats attendus à terme

- Constituer un référentiel maintenu des applications, serveurs, environnements et responsables.
- Réduire le délai d'identification des applications touchées par une alerte de sécurité.
- Structurer la priorisation du risque et l'affectation des mesures correctives.
- Produire des synthèses traçables à partir de données vérifiées et datées.
- Déployer des contrôles uniquement sur des périmètres explicitement autorisés.

02 · Contexte et objectifs

Les équipes qui gèrent des applications PHP ont besoin d'une vue fiable des versions de runtime, dépendances, environnements d'hébergement, résultats d'audit et mesures de remédiation. Un inventaire incomplet transforme une alerte de vulnérabilité en enquête manuelle, parfois urgente. PHP-SHIELD propose de relier ces informations à une gouvernance des accès et à un historique d'opérations.

Objectifs produit

- Inventorier les applications et leur contexte d'hébergement, avec un propriétaire métier et technique.
- Centraliser les versions PHP, SAPIs, extensions et métadonnées utiles aux vérifications autorisées.
- Préparer l'audit des dépendances Composer et l'analyse statique de code PHP.
- Classer les constats avec sévérité, justification, preuve, périmètre et état de traitement.
- Relier les constats aux responsables, échéances, mesures et vérifications de clôture.
- Proposer des exports lisibles par les équipes techniques et les décideurs.

Principes de conception

Privilégier des données vérifiables et des états vides explicites; séparer démonstration et données opérationnelles; appliquer le moindre privilège; rendre les audits déclenchés, autorisés et journalisables; documenter les limites et les dépendances.

03 · Périmètre fonctionnel

Le tableau distingue les éléments d'interface actuellement visibles de capacités encore prévues. Un écran ou un bouton ne constitue pas à lui seul la preuve qu'un moteur métier ou un stockage opérationnel existe.

Domaine	État constaté	Capacité cible
Connexion	Page de connexion, aperçu démo et API d'authentification de fondation.	Comptes nominatifs, habilitations, gestion d'échec et contrôles adaptés à l'environnement.
Vue d'ensemble	Shell visuel; données vides via API et jeu explicitement simulé en démo.	Indicateurs fondés sur l'inventaire et les événements persistés.
Applications	Vue de module et états d'attente.	Registre versionné, propriétaire, criticité, cycle de vie et liens d'environnement.
Infrastructure	Vue de module et états d'attente.	Inventaire de serveurs, environnements et relations avec les applications.
Audits PHP / web	Écrans préparatoires; aucun scan lancé par l'aperçu.	Moteurs et adaptateurs isolés, périmètres autorisés, résultats reproductibles.
Constats & remédiation	Vues préparatoires; absence de résultats réels dans le dépôt actuel.	Enregistrement de preuves, priorisation, affectation, suivi et clôture.
Rapports	Exports PDF et Excel du tableau de posture disponible.	Rapports gouvernés, historisés, vérifiés et adaptés aux rôles.
PWA / interface	Interface installable, thèmes, recherche et navigation adaptative.	Amélioration progressive, tests d'accessibilité et cache hors ligne maîtrisé.

Données de démonstration
Les chiffres affichés en mode aperçu sont illustratifs et signalés comme simulés. Ils ne décrivent pas des serveurs, applications, vulnérabilités ou opérations réels. L'API de fondation retourne un état vide pour le tableau de bord.

04 · Architecture et flux

L'architecture sépare l'interface statique, l'API et le stockage. Cette séparation permet d'héberger l'aperçu public indépendamment du service privé et de ne pas exposer la base de données dans GitHub Pages.

Interface web React · TypeScript · Vite PWA · rapports client	**API applicative** FastAPI · endpoints REST Session HTTP-only	**Stockage relationnel** PostgreSQL SQLAlchemy · Alembic
GitHub Pages Aperçu frontend uniquement	**Service privé** Authentification et API à déployer séparément	**Schéma initial** Utilisateurs · rôles institutions · services

Vue logique de référence — les moteurs d'audit et la persistance métier ne sont pas encore intégrés.

Composants du dépôt

Couche	Technologies et rôle
Frontend	React, TypeScript et Vite; thème clair/sombre, navigation par modules, connexion, aperçu et exports PDF/XLSX.
Backend	FastAPI; santé, authentification de fondation, consultation de session, déconnexion et réponse dashboard vide.
Identité	SQLAlchemy définit utilisateurs actifs, empreinte de mot de passe, rôles, institutions et départements.
Migrations	Alembic fournit la migration initiale des tables d'identité et d'organisation.
Exécution	Docker Compose décrit les services de développement; secrets et paramètres réels restent à configurer.
Publication	GitHub Actions construit l'interface et publie l'aperçu Pages; cela ne déploie pas l'API ni PostgreSQL.

> **Limite de déploiement**
> GitHub Pages ne peut pas exécuter le backend FastAPI ni joindre la base PostgreSQL. L'URL publique sert un aperçu statique. Un environnement institutionnel exigerait une API dédiée, des secrets protégés, un domaine contrôlé et une validation de sécurité.

05 · Données et contrôles de sécurité

La base relationnelle commence par les entités d'identité et d'organisation. Les objets applicatifs métier, les résultats d'audit, les vulnérabilités et les actions de remédiation ne sont pas encore modélisés dans cette version.

Contrôles déjà présents dans la fondation

- Les mots de passe sont vérifiés par empreinte côté serveur; le flux d'administration initial est une commande de bootstrap interactive.
- Les sessions utilisent un cookie signé HTTP-only avec durée de huit heures et SameSite=Lax; l'attribut Secure est activé hors développement.
- L'API autorise les origines CORS explicitement déclarées et exige un secret d'au moins 32 caractères hors développement.
- Aucune inscription publique n'est prévue; la création initiale d'un administrateur est manuelle côté serveur.
- Les journaux, restrictions d'accès métier, MFA et limite distribuée des tentatives demeurent à finaliser avant usage réel.

Socle de données

Entité initiale	Rôle
Utilisateur	Adresse unique, empreinte de mot de passe, état actif et horodatages.
Rôle	Nom de rôle et relation plusieurs-à-plusieurs avec les comptes.
Institution	Organisation de rattachement, avec nom unique.
Département	Service lié à une institution; suppression de l'institution protégée par contrainte.

Règle opérationnelle
Tout contrôle actif doit viser un actif confié ou explicitement autorisé, un périmètre limité, une fenêtre approuvée, un propriétaire averti et un plan de retour arrière. L'aperçu public ne doit jamais être utilisé comme preuve d'un scan réel.

06 · Maturité et feuille de route

Les phases suivantes donnent un ordre de réalisation proposé. Une étape est considérée prête seulement lorsqu'elle a une logique vérifiable, des contrôles d'accès, des validations, un comportement d'erreur documenté et des tests appropriés.

Phase	Contenu envisagé	Critère de sortie
1 · Fondation	Structure, interface, identité visuelle, API, schéma initial, authentification de base, migrations, conteneurs et CI.	Installation reproductible; migration depuis une base vide; tests des parcours et des erreurs.
2 · Cœur PHP-SHIELD	Registre, inventaire, versions PHP, dépendances Composer, premiers constats et priorisation.	Données persistées; permissions; preuves; tests métier; aucun résultat non vérifié.
3 · Analyses avancées	Adaptateurs Semgrep/PHPStan/Psalm/ZAP selon les besoins; exécutions isolées; suivi de remédiation.	Autorisation vérifiée; exécution contenue; traçabilité; gestion des délais et résultats.
4 · Gouvernance	Rapports, exports, audit trail, alertes d'avis de sécurité et règles de conservation.	Données sources réconciliées; attribution; historiques et contrôle de publication.
5 · Préparation pilote	Essais intégrés, sécurité, sauvegarde/restauration, procédures et réception.	Validation technique, procédures approuvées et plan de continuité testé.

Les outils d'analyse sont des pistes d'intégration, pas des capacités actuellement déployées. Leur sélection devra être compatible avec les contraintes d'exploitation, de licence, de confidentialité et de sécurité de l'environnement retenu.

Gestion de version

Le dépôt est la source du code. Les secrets ne doivent pas être versionnés. Les modifications doivent passer par des revues, des contrôles automatisés, un suivi des migrations et une procédure de publication. Un succès GitHub Pages atteste de la publication du frontend, pas d'une mise en service opérationnelle de l'ensemble du produit.

07 · Gouvernance, risques et limites

La plateforme vise un contexte institutionnel sensible. Il faut éviter toute ambiguïté entre proposition, démonstration, pilote autorisé et système accrédité. Les données opérationnelles ne doivent entrer qu'après désignation des responsables, définition de la classification et validation du dispositif d'hébergement.

Risque	Réponse attendue
Maturité surestimée	Étiqueter clairement les écrans en construction, les données démo et les intégrations planifiées.
Accès inadapté	Finaliser les rôles et permissions au niveau métier, MFA selon politique, révocation et journalisation.
Audit hors périmètre	Exiger une autorisation documentée, une cible et une fenêtre; isoler le moteur; limiter les requêtes.
Exposition de données	Chiffrement en transit et au repos, secrets gérés, minimisation, sauvegardes protégées et règles de rétention.
Dépendance à un service	Surveiller les composants, corriger les vulnérabilités, documenter versions et procédure de mise à jour.
Rapport trompeur	Date, origine, version, périmètre, hypothèses et statut de validation affichés sur chaque export.

Périmètre de responsabilité

Les propriétaires des actifs autorisent les contrôles et valident les constats. Les administrateurs gèrent l'environnement et les comptes. Les auditeurs déclenchent les contrôles permis et documentent les preuves. Les décideurs approuvent les priorités et les ressources. Le porteur du projet fournit le produit à évaluer; les institutions compétentes conservent la décision d'adoption, d'hébergement et d'accréditation.

> **Aucune approbation revendiquée**
> Les mentions DRC, DANTIC et Présidence décrivent le contexte de conception visé. Elles ne signifient pas que le produit est une plateforme officielle ou validée par ces institutions.

08 · Préparation d’un pilote

Avant un pilote, réunir le propriétaire métier, le responsable technique, l’administrateur d’infrastructure, le responsable sécurité, le référent protection des données et les personnes chargées de l’exploitation. Le pilote doit être limité à quelques actifs explicitement autorisés et à des données de test maîtrisées.

Préparation technique

- Choisir un hébergement privé pour API et base de données; isoler développement, test et production.
- Configurer HTTPS, secrets uniques et rotation, origines web, sauvegarde chiffrée et surveillance des erreurs.
- Définir rôles, droits sur les actifs, processus d’arrivée/départ et revue périodique des accès.
- Établir les règles de collecte, classification, durée de conservation et accès aux preuves.
- Définir autorisations de scan, limites de débit, plages horaires, coordonnées d’urgence et procédure d’arrêt.
- Valider une restauration de sauvegarde et la reprise après indisponibilité de l’API ou de la base.
- Tester les migrations depuis une base vide puis depuis une version antérieure réaliste.

Décisions à obtenir avant le pilote

Décision	Évidence attendue
Autorisation de périmètre	Liste des applications, environnements, propriétaires, plages et opérations permises.
Gouvernance des données	Classification, responsabilités, conservation, localisation et règles d’accès.
Modèle d’exploitation	Hébergement, supervision, sauvegardes, mises à jour, astreinte et plan d’incident.
Réception	Tests signés, seuils de qualité, limites reconnues et mécanisme de désactivation.

09 · Indicateurs de réussite

Les indicateurs devront être calculés à partir d'enregistrements horodatés et contrôlés; les valeurs d'exemple de la démonstration ne sont pas des résultats de référence.

Indicateur	Définition proposée	Usage
Couverture d'inventaire	Actifs connus et validés / périmètre déclaré.	Identifier les angles morts et les écarts de registre.
Délai d'affectation	Temps entre un constat validé et l'assignation d'un responsable.	Mesurer la réactivité de gouvernance.
Délai de correction	Temps de la validation au correctif confirmé, ventilé par sévérité.	Prioriser les risques et améliorer la coordination.
Âge des constats	Nombre et durée des constats encore ouverts par niveau.	Repérer les retards et les besoins d'escalade.
Taux de vérification	Actions corrigées dont le résultat est contrôlé après changement.	Éviter une clôture uniquement déclarative.
Qualité des données	Part des actifs avec responsable, version et date de révision connus.	Évaluer la fiabilité de l'inventaire.

Aucun seuil universel ne doit être inventé. Les responsables métier et sécurité fixeront les objectifs selon le risque, les contraintes de service et la politique en vigueur.

Conclusion

PHP-SHIELD RDC propose une trajectoire cohérente pour structurer la sécurité des applications PHP, à condition de transformer le socle actuel en produit gouverné, éprouvé et exploité. La priorité suivante est d'achever le registre et son modèle de permissions, puis de connecter un premier parcours de données contrôlé, avec migrations et tests d'intégration.

10 · Glossaire

Terme	Définition
Actif	Application, service ou infrastructure inscrite au périmètre de gestion.
Audit autorisé	Analyse lancée avec l'accord du propriétaire, sur un périmètre et une fenêtre définis.
Constat	Observation de sécurité appuyée par une preuve et un contexte de validation.
CVE	Identifiant public de référence d'une vulnérabilité divulguée.
Composer	Gestionnaire de dépendances PHP et manifeste du graphe de composants.
MFA	Authentification multifacteur; son activation n'est pas encore intégrée à la fondation décrite.
Migration	Évolution versionnée et reproductible du schéma de base de données.
PWA	Application web progressive installable; elle n'embarque pas l'API privée.
SAST	Analyse statique du code source sans exécution de l'application cible.
SCA	Analyse des composants et dépendances logiciels.
Remédiation	Action documentée visant à corriger ou réduire un risque confirmé.
Périmètre	Ensemble explicite des actifs, opérations et limites autorisés pour une analyse.

Propriété du projet
PHP-SHIELD RDC est présenté dans ce dossier comme un projet fondé et détenu par LOKALA LOMBOTO Jonathan. Toute décision d'adoption ou de déploiement institutionnel reste soumise à l'évaluation des autorités compétentes.